



UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

Department of Cyber Security Engineering

Project Report

Name of the project : Zero Trust Access Control System
Course Name : Object Oriented Programming Language Sessional
Course Code : PROG 112

Instructor :

Name	Designation	Department
Md Masud Rana	Lecturer	Cyber Security Engineering

Team Members :

Name	Student ID	level	Term
Mst. Bithi Khatun	2404024	01	02
Afra Anjum Odhora	2404025	01	02

Abstract

The Zero Trust Access Control System is a security prototype developed in C++ using object-oriented programming concepts. It demonstrates the principle of **Never Trust, Always Verify** by requiring continuous authentication and authorization before granting access to any resource.

The system enforces Multi-Factor Authentication (MFA) combining password verification and a one-time password (OTP), Role-Based Access Control (RBAC) across three user tiers (Admin, User, Guest), thread-based session management with automatic expiration, re-authentication for sensitive actions, real-time suspicious activity detection, and an XOR-encrypted audit logging mechanism.

The project was developed using Object-Oriented Programming concepts including encapsulation, abstraction, composition, constructors, STL containers, multithreading, lambda expressions, and file I/O.

Chapter 1: Introduction

1.1 Background

In modern digital environments, organizations face increasing threats such as credential theft, unauthorized access, and session hijacking. Traditional security models often verify users only once at login, which is no longer sufficient for protecting sensitive resources. The proposal identifies Bangladesh's rapid digital transformation, especially in banking, online education, and e-governance, as a major reason for stronger security mechanisms.

1.2 Problem Statement

1. Brute-force attacks exploit weak or reused passwords.
2. Phishing attacks steal credentials, making security controls ineffective.
3. There is no continuous verification.
4. A logged-in user is trusted for the entire session.
5. Audit logs, when they exist, are often stored in plaintext.

1.3 Objectives

The main objectives of this project are:

- Implement multi-factor authentication using password and OTP.
- Apply role-based access control for Admin, User, and Guest roles.
- Enforce login attempt limits to reduce brute-force attacks.
- Add session management with automatic expiration after inactivity.
- Require re-authentication for sensitive or expired access requests.
- Detect suspicious activity from repeated failures and denied requests.
- Store security events in an encrypted audit log.

Chapter 2: Workflow

1. User enters username.
2. System verifies whether the username exists.
3. User enters password.
4. System generates and checks OTP.
5. If authentication succeeds, session starts.
6. Role-based menu is displayed according to user role.
7. User selects an action such as file access, database access, or public viewing.
8. System checks session validity, re-authentication requirement, and access rights.
9. Access is granted or denied, and the event is recorded in the log file.
10. If repeated failures occur, suspicious activity alerts are shown.

Chapter 3: System Design

3.1 Class Design

The system is built around three primary classes:

Class	Attributes	Responsibility
User	name, pass, role (public); User(n,p,r) constructor	Represents a system user with identity and role attributes.
Logger	encrypt(s) [private]; write(msg) [public]	Encrypts and persists all audit events to audit.log.

Session	active [public bool]; start() launches timer thread	Manages session lifecycle and automatic inactivity expiry.
---------	---	--

Chapter 4: Output

4.1 Scenario A – Successful Admin Login & Access

```

Username: admin
Password: 123
OTP: 5221
Enter OTP: 5221
Login Success

===== MENU =====
Role : Admin
1. Access File
2. Access Database
3. View Security Logs
4. Logout
Choice: 1
Access Granted

===== MENU =====
Role : Admin
1. Access File
2. Access Database
3. View Security Logs
4. Logout
Choice: 2
Access Granted

```

4.2 Scenario B – Account Lockout after 3 Failed Attempts

```

Username: user
Password: 123
Wrong Password
Password: 123
Wrong Password
Password: 123
Wrong Password
Account Locked!

```

4.3 Scenario C – Session Expiry & Re-authentication

```
===== MENU =====
Role : Admin
1. Access File
2. Access Database
3. View Security Logs
4. Logout
Choice:
Session Expired!
1
Re-authentication Required
Enter Password Again: 123
Re-authentication Success
Access Granted
```

4.4 Suspicious Activity Detected

```
Username: user
Password: 111
OTP: 3844
Enter OTP: 3844
Login Success

===== MENU =====
Role : User
1. Access File
2. Access Database
3. Logout
Choice: 2
Access Denied
Risk Level : LOW

===== MENU =====
Role : User
1. Access File
2. Access Database
3. Logout
Choice: 2
Access Denied
Suspicious Activity Detected!
Risk Level : LOW
```

Chapter 5: OOP Concepts Used

This project uses several object-oriented programming concepts effectively.

OOP Concept	Description
Class & Object	Each entity is modelled as a class with its own attributes and behaviors.
Encapsulation	The encrypt() method is private; external code cannot access it directly.
Constructor	User attributes are initialized via a parameterized constructor at object creation.
Abstraction	Callers invoke write () without knowing the XOR encryption logic inside.
Composition	Session and Logger objects are created inside main(), forming a has a relationship.
STL – Vector	All user records are stored and iterated using a standard vector.
STL – Map	Login-attempt counters and deny counters are tracked with maps keyed by username.
Thread / Concurrency	A detached thread runs the inactivity timer independently of the main thread.
Lambda Expression	An anonymous lambda captures the session pointer for the timer thread.
File I/O	All events are appended to an external encrypted log file using fstream.

Chapter 6: Limitations and Improvement

6.1 Limitations

The current system is a prototype and uses hardcoded users, so it is not connected to a real database. The OTP is displayed on screen instead of being sent through email or SMS, which makes it suitable for demonstration but not for deployment. The encryption method for logs is a simple XOR-based approach, which is weak for real-world security. The system also lacks persistent session storage, advanced password hashing, and proper exception handling for invalid input.

6.2 Improvements

Future versions can include a database-driven user system, secure password hashing, and real OTP delivery methods. The logging system can be upgraded to strong cryptographic encryption instead of XOR. The session system can also be improved with multi-user concurrency, better timeout control, and more detailed risk scoring.

Chapter 7: Conclusion

The Zero Trust Access Control System is a security prototype developed in C++. It successfully demonstrates a secure login and authorization workflow based on continuous verification. It shows how MFA, RBAC, session control, re-authentication, and logging can work together to strengthen system security.